

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

07

자바의 입출력



07-4 예외 처리

07-5 스레드

07-6 함수형 프로그래밍

07-4 예외 처리

▪ 예외는 언제 발생하는가?

- 존재하지 않는 파일을 열려고 시도할 때 FileNotFoundException 예외 발생

Do it! 코딩해 보세요 Sample.java

```
import java.io.*;

public class Sample {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new FileReader("나없는파일"));
        br.readLine();
        br.close();
    }
}
```

실행 결과

```
Exception in thread "main" java.io.FileNotFoundException: 나없는파일 (지정된 파일을 찾을 수 없습니다)
    at java.io.FileInputStream.open(Native Method)
    at java.io.FileInputStream.<init>(Unknown Source)
    at java.io.FileInputStream.<init>(Unknown Source)
    at java.io.FileReader.<init>(Unknown Source)
    ...
```

07-4 예외 처리

예외는 언제 발생하는가?

- 산술에 문제가 생겼을 때 ArithmeticException 예외 발생

Do it! 코딩해 보세요

```
public class Sample {
    public static void main(String[] args){
        int c = 4 / 0;
    }
}
```

실행 결과

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Sample.main(Sample.java:14)
```

07-4 예외 처리

예외는 언제 발생하는가?

- 배열에서 아무것도 없는 곳을 가리켰을 때 ArrayIndexOutOfBoundsException 예외 발생

Do it! 코딩해 보세요

```
public class Sample {
    public static void main(String[] args){
        int[] a = {1, 2, 3};
        System.out.println(a[3]);
    }
}
```

실행 결과

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 3
    at Sample.main(Sample.java:17)
```

07-4 예외 처리

▪ 예외 처리하기

▪ 숫자를 0으로 나누었을 때 발생하는 예외 처리 – try ~ catch 문 사용

Do it! 코딩해 보세요

```
public class Sample {
    public static void main(String[] args) {
        int c;
        try {
            c = 4 / 0;
        } catch (ArithmeticException e) {
            c = -1;  // 예외가 발생하여 이 문장이 수행된다.
        }
    }
}
```

```
try {
    <수행할 문장 1>;
    <수행할 문장 2>;
    ...
} catch(예외1) {
    <수행할 문장 A>;
    ...
} catch(예외2) {
    <수행할 문장 a>;
    ...
}
```

07-4 예외 처리

finally

- 어떤 예외가 발생하더라도 반드시 실행되어야 하는 부분이 있을 때 사용
- finally 문은 try 문장 수행 중 예외 발생 여부에 상관없이 무조건 실행됨

Do it! 코딩해 보세요

```
public class Sample {
    public void shouldBeRun() {
        System.out.println("ok, thanks");
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        int c;
        try {
            c = 4 / 0;
        } catch (ArithmeticException e) {
            c = -1;
        } finally {
            sample.shouldBeRun();  // 예외에 상관없이 무조건 수행된다.
        }
    }
}
```

실행 결과

```
ok, thanks
```

07-4 예외 처리

▪ 예외 활용하기 - RuntimeException과 Exception

▪ RuntimeException

- 실행 시 발생하는 예외 – 발생할 수도 발생하지 않을 수도 있는 경우에 사용(Checked Exception)

▪ 예외 발생 예

Do it! 코딩해 보세요

```
class FoolException extends RuntimeException {
}

public class Sample {
    public void sayNick(String nick) {
        if ("바보".equals(nick)) {
            throw new FoolException();
        }
        System.out.println("당신의 별명은 "+nick+" 입니다.");
    }
    public static void main(String[] args) {
        Sample sample = new Sample();
        sample.sayNick("바보");
        sample.sayNick("야호");
    }
}
```

실행 결과

```
Exception in thread "main" FoolException
    at Sample.sayNick(Sample.java:7)
    at Sample.main(Sample.java:14)
```

07-4 예외 처리

▪ 예외 활용하기 - RuntimeException과 Exception

▪ Exception

- ▪ 컴파일 시 발생하는 예외 – 예측이 가능한 경우에 사용(Unchecked Exception)

▪ 예외 발생 예

Do it! 코딩해 보세요

```
class FoolException extends Exception {
}

public class Sample {
    public void sayNick(String nick) {
        try {
            if ("바보".equals(nick)) {
                throw new FoolException();
            }
            System.out.println("당신의 별명은 "+nick+" 입니다.");
        }catch(FoolException e) {
            System.err.println("FoolException이 발생했습니다.");
        }
    }

    public static void main(String[] args) {
        Sample sample = new Sample();
        sample.sayNick("바보");
        sample.sayNick("야호");
    }
}
```

07-4 예외 처리

▪ 예외 던지기

- throws라는 구문을 이용해 예외를 뒤로 미룰 수 있음

Do it! 코딩해 보세요

```
class FoolException extends Exception {
}

public class Sample {
    public void sayNick(String nick) throws FoolException {
        if ("바보".equals(nick)) {
            throw new FoolException();
        }
        System.out.println("당신의 별명은 "+nick+" 입니다.");
    }
    public static void main(String[] args) {
        Sample sample = new Sample();
        try {
            sample.sayNick("바보");
            sample.sayNick("야호");
        }catch (FoolException e) {
            System.err.println("FoolException이 발생했습니다.");
        }
    }
}
```

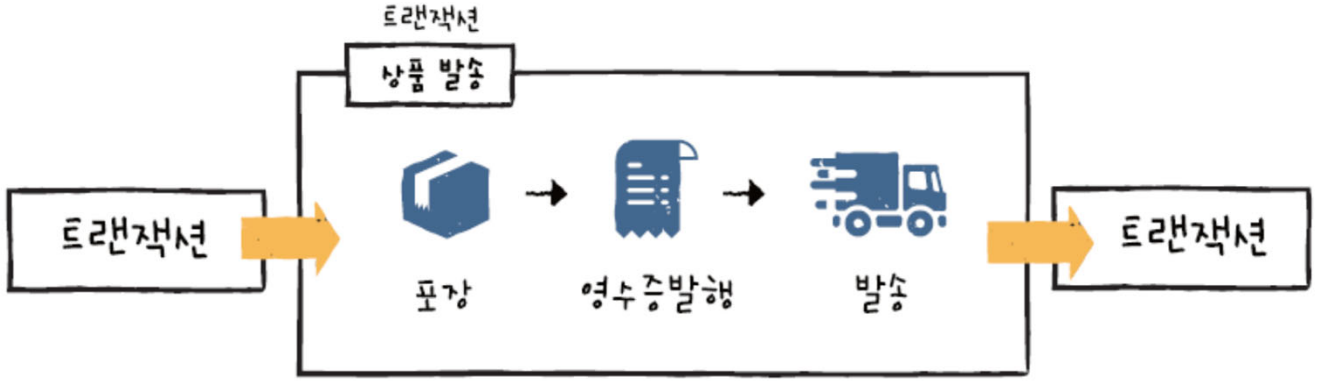
실행 결과

```
FoolException이 발생했습니다.
```

- main 메서드에서 try ~ catch 문으로 sayNick 메서드의 FoolException 예외 처리

07-4 예외 처리

- **트랜잭션(transaction)**
  - 쇼핑몰의 '상품발송'이라는 트랜잭션에 있는 작업들



  - 쇼핑몰 운영자는 이 3가지 일 중 하나라도 실패하면 3가지 모두 취소하고 '상품발송' 전 상태로 되돌리고 싶어 함

07-4 예외 처리

- **트랜잭션(transaction)**
  - 쇼핑몰의 트랜잭션 슈도코드(pseudocode) 작성 예

```
상품발송() {
   try {
      포장();
      영수증발행();
      발송();
   }catch(예외) {
      모두취소();  // 하나라도 실패하면 모두 취소한다.
   }
}
```

```
포장() throws 예외 {
   ...
}

영수증발행() throws 예외 {
   ...
}

발송() throws 예외 {
   ...
}
```

    - 포장, 영수증발행, 발송 작업 중 하나라도 실패할 경우 예외가 발생되어 상품발송이 모두 취소됨

07-5 스레드

- **Thread**
  - 스레드(thread)를 이용하면 한 프로세스 내에서 두 가지 또는 그 이상의 일을 동시에 할 수 있음
  - 스레드는 순서에 상관없이 동시에 실행됨

07-5 스레드

Thread

총 10개의 스레드를 실행시키는 예

Do it! 코딩해 보세요

```
public class Sample extends Thread {
    int seq;

    public Sample(int seq) {
        this.seq = seq;
    }

    public void run() {
        System.out.println(this.seq + " thread start."); // 스레드 시작
        try {
            Thread.sleep(1000); // 1초 대기한다.
        } catch (Exception e) {
        }
        System.out.println(this.seq + " thread end."); // 스레드 종료
    }

    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) { // 총 10개의 스레드를 생성하여 실행한다.
            Thread t = new Sample(i);
            t.start();
        }
        System.out.println("main end."); // main 메서드 종료
    }
}
```

실행 결과

```
0 thread start.
4 thread start.
6 thread start.
2 thread start.
main end.
3 thread start.
7 thread start.
8 thread start.
1 thread start.
9 thread start.
5 thread start.
0 thread end.
4 thread end.
2 thread end.
6 thread end.
7 thread end.
3 thread end.
8 thread end.
9 thread end.
1 thread end.
5 thread end.
```

07-5 스레드

- **Join**
  - 스레드가 종료될 때까지 기다리게 하는 메서드
  - 스레드가 모두 종료된 후 그다음 작업을 진행해야 할 때 사용
  - 코드 예시(다음 화면)

07-5 스레드

Join

Do it! 코딩해 보세요

```
import java.util.ArrayList;
public class Sample extends Thread {
    int seq;
    public Sample(int seq) {
        this.seq = seq;
    }

    public void run() {
        System.out.println(this.seq+" thread start.");
        try {
            Thread.sleep(1000);
        }catch(Exception e) {
        }
        System.out.println(this.seq+" thread end.");
    }
```

```
    public static void main(String[] args) {
        ArrayList<Thread> threads = new ArrayList<>();
        for(int i=0; i<10; i++) {
            Thread t = new Sample(i);
            t.start();
            threads.add(t);
        }

        for(int i=0; i<threads.size(); i++) {
            Thread t = threads.get(i);
            try {
                t.join(); // t 스레드가 종료할 때까지 기다린다.
            }catch(Exception e) {
            }
        }
        System.out.println("main end.");
    }
}
```

실행 결과

```
0 thread start.
5 thread start.
2 thread start.
6 thread start.
9 thread start.
1 thread start.
7 thread start.
3 thread start.
8 thread start.
4 thread start.
0 thread end.
5 thread end.
2 thread end.
9 thread end.
6 thread end.
1 thread end.
7 thread end.
4 thread end.
8 thread end.
3 thread end.
main end
```

07-5 스레드

- **Runnable**
  - Thread 객체가 Thread 클래스를 상속하면 다른 클래스를 상속할 수 없지만, Runnable 인터페이스를 사용하면 다른 클래스 상속이 가능하므로 유연한 프로그램으로 만들 수 있음
  - 코드 예시(다음 화면)

07-5 스레드

▪ Runnable

Do it! 코딩해 보세요

```
import java.util.ArrayList;

public class Sample implements Runnable {
    int seq;
    public Sample(int seq) {
        this.seq = seq;
    }

    public void run() {
        System.out.println(this.seq+" thread start.");
        try {
            Thread.sleep(1000);
        }catch(Exception e) {
        }
        System.out.println(this.seq+" thread end.");
    }
```

```
    public static void main(String[] args) {
        ArrayList<Thread> threads = new ArrayList<>();
        for(int i=0; i<10; i++) {
            Thread t = new Thread(new Sample(i));
            t.start();
            threads.add(t);
        }

        for(int i=0; i<threads.size(); i++) {
            Thread t = threads.get(i);
            try {
                t.join();
            }catch(Exception e) {
            }
        }
        System.out.println("main end.");
    }
}
```

07-6 함수형 프로그래밍

- **람다(lambda)**
  - **일반적인 코드**
    - 두 개의 정수를 입력으로 받아 정수의 결괏값을 리턴하는 sum 함수를 정의한 인터페이스

Do it! 코딩해 보세요

```
interface Calculator {
    int sum(int a, int b);
}

class MyCalculator implements Calculator {
    public int sum(int a, int b) {
        return a+b;
    }
}
```

```
public class Sample {
    public static void main(String[] args) {
        Calculator mc = new MyCalculator();
        int result = mc.sum(3, 4);
        System.out.println(result);  // 7 출력
    }
}
```

실행 결과

```
7
```

07-6 함수형 프로그래밍

- **람다(lambda)**
  - **람다를 적용한 코드**

Do it! 코딩해 보세요

```
interface Calculator {
    int sum(int a, int b);
}

public class Sample {
    public static void main(String[] args) {
        Calculator mc = (int a, int b) -> a +b;
        int result = mc.sum(3, 4);
        System.out.println(result);
    }
}
```

← 람다를 적용한 코드



실행 결과

```
7
```

- 람다 함수를 사용하면 MyCalculator와 같은 실제 클래스 없이도 Calculator 객체를 생성할 수 있고, 일반적인 코드보다 훨씬 간단해짐

07-6 함수형 프로그래밍

람다(lambda)

인터페이스 사용 시 주의 사항

- 인터페이스의 메서드가 1개 이상이면 람다 함수를 사용할 수 없음
- 람다 함수로 사용할 인터페이스는 @FunctionalInterface 어노테이션을 사용하는 것이 좋음
  - @FunctionalInterface는 인터페이스가 함수형 인터페이스임을 표시
  - 단 하나의 메서드만 가질 수 있게 하는 역할

```
@FunctionalInterface
interface Calculator {
    int sum(int a, int b);
    int mul(int a, int b);  // @FunctionalInterface는 두 번째 메서드를 허용하지 않는다.
}
```

07-6 함수형 프로그래밍

람다(lambda)

람다 축약하기

- 람다를 적용한 자바 코드는 조금 더 축약이 가능함
- 인터페이스에 이미 입출력에 대한 타입이 정의되어 있으므로 입력값의 자료형인 int를 생략할 수 있음

Do it! 코딩해 보세요

```
interface Calculator {
    int sum(int a, int b);
}

public class Sample {
    public static void main(String[] args) {
        Calculator mc = (a, b) -> a +b;
        int result = mc.sum(3, 4);
        System.out.println(result);
    }
}
```

07-6 함수형 프로그래밍

람다(lambda)

람다 함수 인터페이스

- 인터페이스를 생성하는 대신 함수형 프로그래밍을 위해 제공되는 인터페이스를 사용하여 코드를 축약할 수 있음

Do it! 코딩해 보세요

```
import java.util.function.BinaryOperator;
public class Sample {
    public static void main(String[] args) {
        BinaryOperator<Integer> mc = (a, b) -> a + b;
        int result = mc.apply(3, 4);
        System.out.println(result);  // 7 출력
    }
}
```

07-6 함수형 프로그래밍

- **스트림(stream)**
  - 데이터가 물결처럼 흘러가면서 필터링 과정을 통해 여러 번 변경되어 반환되기 때문에 갖게 된 이름
  - 일반적인 코드보다 간결하고 가독성이 좋음

Do it! 코딩해 보세요

```
import java.util.Arrays;
import java.util.Comparator;

public class Sample {
    public static void main(String[] args) {
        int[] data = {5, 6, 4, 2, 3, 1, 1, 2, 2, 4, 8};
        int[] result = Arrays.stream(data)  // IntStream을 생성한다.
                .boxed()  // IntStream을 Stream<Integer>로 변경한다.
                .filter((a) -> a % 2 == 0)  // 짝수만 뽑아낸다.
                .distinct()  // 중복을 제거한다.
                .sorted(Comparator.reverseOrder())  // 역순으로 정렬한다.
                .mapToInt(Integer::intValue)  // Stream<Integer>를 IntStream으로 변경한다.
                .toArray()  // int[] 배열로 반환한다.
                ;
    }
}
```



감사합니다.